

Athenaeum

Library Management System — MySQL Implementation

Database Management Systems · Capstone Project

Project Report & Technical Documentation

Backend: MySQL 8.0 · Frontend: HTML / CSS / JS + Express

Hosted on Vercel (app) + Aiven (database)

Contents

1. Overview & Objectives
2. System Architecture (3-tier)
3. Entity–Relationship Design
4. Schema & Constraints (`sql/01_schema.sql`)
5. Normalization (1NF → 3NF)
6. Data Dictionary
7. Sample Data
8. Views, Queries, Triggers & Procedures
9. Frontend & Backend Integration
10. Deployment (Docker · Aiven · Vercel)
11. DBMS Concepts Reference
12. Testing
13. Viva Questions & Answers

1. Overview & Objectives

Athenaeum is a complete Library Management System on a normalized MySQL database. It models real library operations: cataloguing books, tracking each physical copy, registering members and staff, issuing/returning books, charging late fines, and handling reservations.

- Relational schema in **3NF** with full integrity constraints.
- All relationship types — **1:M**, **M:N**, **recursive**, **1:1**.
- Advanced SQL — joins, subqueries, aggregates, **GROUP BY/HAVING**, views, indexes.
- Integrity in the database — triggers, stored procedures, transactions.
- A working frontend connected to the **live database**, **hosted online**.

2. System Architecture (3-tier)

A browser cannot connect to MySQL directly, so the system uses three tiers:

```
Browser (HTML/CSS/JS)  -->  Express API (Node)  -->  MySQL (library_db)
  fetch('/api/...')      views + stored procs    tables + triggers
  public/                api/                    Aiven (cloud)
```

- **Presentation** — vanilla front-end in **public/**.
- **Application** — Express in **api/**; calls views and stored procedures, returns JSON.
- **Data** — MySQL holds the schema and all integrity logic.

3. Entity–Relationship Design

Relationship	Type	Implemented by
Publisher → Book	1 : M	book.publisher_id
Category → Book	1 : M	book.category_id
Category → Category	1 : M recursive	category.parent_id
Book ↔ Author	M : N	book_author junction
Book → BookCopy	1 : M	book_copy.book_id
BookCopy → Borrowing	1 : M	borrowing.copy_id
Member → Borrowing	1 : M	borrowing.member_id
Staff → Borrowing	1 : M	borrowing.issued_by
Borrowing → Fine	1 : 1	fine.borrow_id UNIQUE
Book/Member → Reservation	1 : M	reservation FKs

Key design choice: `book` (title) is separated from `book_copy` (physical item), so one title can have many copies, each with its own status.

4. Schema & Constraints (`sql/01_schema.sql`)

11 InnoDB tables. Constraint types used: **PRIMARY KEY**, **FOREIGN KEY** (with RESTRICT / CASCADE / SET NULL), **UNIQUE** (isbn, barcode, email), **NOT NULL**, **DEFAULT**, **CHECK** (price ≥ 0, due ≥ borrow, birth-year range), and **ENUM** for status fields. Indexes are added on searched/filtered columns; an `ALTER TABLE` adds an audit timestamp.

```
CREATE TABLE book (  
  book_id INT AUTO_INCREMENT PRIMARY KEY, isbn CHAR(13) NOT NULL UNIQUE,  
  title VARCHAR(200) NOT NULL, publisher_id INT NOT NULL, category_id INT NOT NULL,  
  price DECIMAL(8,2) CHECK(price >= 0),  
  FOREIGN KEY(publisher_id) REFERENCES publisher(publisher_id) ON DELETE RESTRICT,  
  FOREIGN KEY(category_id) REFERENCES category(category_id) ON DELETE RESTRICT  
) ENGINE=InnoDB;
```

5. Normalization (1NF → 3NF)

1NF

Atomic values only. Multi-valued authors and multiple copies moved to `book_author` and `book_copy`.

2NF

No partial dependency. In `book_author` (composite key `book_id, author_id`), the only attribute `author_role` depends on the whole key.

3NF

No transitive dependency. Publisher city lives in `publisher`, category data in `category`, fine data in `fine` — each non-key attribute depends only on its key.

Result: every non-key attribute depends on the key, the whole key, and nothing but the key. No redundancy, no insert/update/delete anomalies — the schema is in 3NF.

6. Data Dictionary (key tables)

book

Column	Type	Constraints
book_id	INT	PK, AUTO_INCREMENT
isbn	CHAR(13)	NOT NULL, UNIQUE
title	VARCHAR(200)	NOT NULL
publisher_id / category_id	INT	NOT NULL, FK
price	DECIMAL(8,2)	CHECK ≥ 0

borrowing

Column	Type	Constraints
borrow_id	INT	PK
copy_id / member_id	INT	NOT NULL, FK
due_date	DATE	NOT NULL, CHECK $\geq$ borrow_date
return_date	DATE	NULL = still on loan

7. Sample Data

~200 records: 10 publishers, 12 categories, 20 authors, 25 books, ~24 author links, 40 copies, 20 members, 5 staff, 30 borrowings, 7 fines, 8 reservations. Loans include returned, active, and overdue cases so the views and fine logic show real results.

8. Views, Queries, Triggers & Procedures

Views (`sql/03_views.sql`)

`vw_book_catalogue` (authors merged via `GROUP_CONCAT`), `vw_book_availability`, `vw_active_loans` (with `DATEDIFF` days overdue), `vw_outstanding_fines`.

Queries (`sql/04_queries.sql`)

Inner/left/self joins; aggregates with `GROUP BY ... HAVING`; scalar, correlated (EXISTS) and NOT IN subqueries; UPDATE/DELETE inside a transaction.

Triggers & procedures (`sql/05_procedures.sql`)

```
-- block issuing an unavailable copy
CREATE TRIGGER trg_borrow_before_insert BEFORE INSERT ON borrowing ...
  IF v_status <> 'Available' THEN SIGNAL SQLSTATE '45000' ...;

-- on late return, raise a fine automatically
CREATE TRIGGER trg_borrow_after_update AFTER UPDATE ON borrowing ...
  IF DATEDIFF(NEW.return_date,NEW.due_date) > 0 THEN INSERT INTO fine ...;

-- transactional issue, with rollback on error
CREATE PROCEDURE sp_issue_book(...) START TRANSACTION; ... COMMIT;
```

9. Frontend & Backend Integration

The front-end (`public/`) is a warm, editorial single-page UI — a script logo, serif headings, a hero banner, and clean tables. JavaScript calls the API with `fetch()` and renders JSON. Issue/return go through stored procedures, so triggers fire automatically and their error messages (e.g. "Copy not available") are shown to the user — genuine frontend–backend–database integration.

```
app.get('/api/books', ... 'SELECT * FROM vw_book_catalogue ...');
app.post('/api/loans', ... 'CALL sp_issue_book(?,?,?,?,@bid)');
```

10. Deployment (Docker · Aiven · Vercel)

Local (Docker)

```
docker compose up -d # MySQL + auto-loads sql/
npm start            # http://localhost:3000
```

Database — Aiven (cloud MySQL)

```
docker run --rm -i -v "$PWD/sql:/s" mysql:8.0 \
  sh -c "mysql -h HOST -P PORT -u avnadmin -pPASS --ssl-mode=REQUIRED < /s/init.sql"
```

App — Vercel (serverless)

`vercel.json` serves `public/` statically and routes `/api/*` to the serverless function `api/index.js`.
The database connection is supplied as environment variables, never hard-coded:

```
MYSQL_URL = mysql://avnadmin:****@HOST:PORT/library_db
DB_SSL    = true
```

The pool in `api/db.js` reads `MYSQL_URL / DB_*` from the environment and falls back to the local Docker defaults — the same code runs locally and in the cloud.

11. DBMS Concepts Reference

Concept	Where it appears
Keys: super, candidate, primary, foreign, composite, surrogate	all <code>*_id</code> , <code>book_author</code> , <code>isbn</code>
Integrity: entity, referential, domain	PK, FK, CHECK/ENUM
SQL sub-languages: DDL / DML / DCL / TCL	schema / data / privileges / transactions
JOINS: inner, left, self	<code>sql/04_queries.sql</code>
Subqueries: scalar, correlated, IN	<code>sql/04_queries.sql</code>
Normal forms: 1NF, 2NF, 3NF, BCNF	Section 5
ACID transactions	<code>sp_issue_book</code> , <code>sp_return_book</code>
Index / View / Trigger / Procedure / Function	Section 8

12. Testing

#	Test	Expected
1	Issue an available copy	Loan created; copy → Borrowed (trigger)
2	Issue a borrowed copy	Rejected by before-insert trigger
3	Return late	Copy freed; fine auto-created
4	Duplicate ISBN	Rejected by UNIQUE
5	price = -50	Rejected by CHECK
6	Delete publisher with books	Rejected by ON DELETE RESTRICT

13. Viva Questions & Answers

Q. Why 3NF?

It removes redundancy and anomalies; every non-key attribute depends only on the primary key, so each fact is stored once.

Q. Where is the M:N relationship?

Book ↔ Author, via the junction `book_author` with composite PK `(book_id, author_id)`.

Q. Recursive relationship?

`category.parent_id` references `category.category_id`.

Q. How is 1:1 made?

A UNIQUE foreign key — `fine.borrow_id`.

Q. Why separate book and book_copy?

To store title facts once and track each physical copy's status separately.

Q. WHERE vs HAVING?

WHERE filters rows before grouping; HAVING filters groups after aggregation.

Q. What are ACID properties and where are they used?

Atomicity, Consistency, Isolation, Durability — in the issue/return stored procedures wrapped in transactions with rollback.

Q. How is integrity enforced without the app?

Constraints (PK/FK/UNIQUE/CHECK/ENUM) plus triggers — e.g. blocking an unavailable copy and auto-creating a late fine.

Q. Why can't the browser talk to MySQL directly?

No DB driver and the security sandbox forbid it; a backend (Express on Vercel) sits in between.

Q. How is it hosted?

The database runs on Aiven (cloud MySQL); the app runs as serverless functions on Vercel, connected via the `MYSQL_URL` environment variable.

Q. ON DELETE CASCADE vs RESTRICT vs SET NULL?

CASCADE deletes children with the parent; RESTRICT blocks deletion while children exist; SET NULL clears the child's FK. We use CASCADE for junctions, RESTRICT to protect loan history, SET NULL for optional issuing staff.

— End of Report —